$F2 \parallel WC_{\max}$ Structural Properties and Dynamic Programming

Author One[a,*], Author Two[b]

[a]Institution One, Location, Country

[b]Institution Two, Location, Country

Abstract

Minimizing $WC_{\max}$ on a two-machine flow shop, denoted $F2 \parallel WC_{\max}$, is a classical NP-hard scheduling problem. Existing exact methods rely on enumerative techniques that scale poorly with instance size, while approximation algorithms often impose restrictive assumptions on processing times. We first sequence all jobs globally by Johnson's rule, obtaining an initial schedule π. Scanning π from left to right, we partition it into at most K blocks: whenever a weight not seen before appears, it serves as the final job of the current block, and the next job starts a new block. We establish structural properties of optimal schedules: the permutation property, the positional constraint that block $\mathcal{G}_k$ jobs cannot appear beyond the first k blocks, and the block-end condition requiring the final job of each block to carry that block's weight (automatically satisfied by construction). The intra-block Johnson ordering inherited from π is further formalized within the DP framework. Building on these properties and the interval-based dynamic programming framework (Hall, 1994) developed for $F2|r_j|C_{\max}$, we design a polynomial-time dynamic programming algorithm with $O(|Y|_{\max} \cdot n)$ complexity that appends jobs block by block. We further show that geometric rounding of weights and processing times converts this algorithm into a polynomial-time approximation scheme.

Keywords: Flow shop scheduling; $WC_{\max}$; Dynamic programming; Structural properties

1 Introduction

2 Preliminaries

We consider the problem $F2 \parallel WC_{\max}$. Let $\mathcal{J} = \{J_1, J_2, \ldots, J_n\}$ be the set of n jobs to be processed on two machines M_1 and M_2 in series: each job J_j must be processed first on M_1 for

*Corresponding author. E-mail address: author@email.com

a_j time units and then on M_2 for b_j time units, with no preemption allowed. All processing times a_j and b_j are assumed to be positive integers. M_1 and M_2 can process at most one job at a time. Each job J_j has a weight $w_j > 0$. A schedule $\pi = (\pi(1), \pi(2), \ldots, \pi(n))$ specifies the processing order on both machines. For a schedule π, let $A_j = \sum_{h=1}^{j} a_{\pi(h)}$ and $B_j = \sum_{h=1}^{j} b_{\pi(h)}$ be the cumulative M_1 and M_2 completion times after the first j jobs. The objective is to find a schedule π that minimizes $WC_{\max}$.

The problem $F2 \parallel WC_{\max}$ can be formulated as a mixed integer linear program (MILP). For a permutation schedule, let the positions be indexed by $k = 1, \ldots, n$. We introduce the following decision variables:

- $x_{jk} = 1$ if job J_j processing at position k, and 0 otherwise;
- C_k^1 completion time of the job at position k on M_1;
- C_k^2 completion time of the job at position k on M_2;
- C_j completion time of job J_j on M_2;
- Z the maximum weighted completion time $WC_{\max}$.
- $M = 2\sum_{j=1}^{n}(a_j + b_j)$ be a sufficiently large constant.

The MILP model is:

$$\min \quad Z \tag{1}$$

$$\text{s.t.} \quad \sum_{k=1}^{n} x_{jk} = 1, \quad j = 1, \ldots, n \tag{2}$$

$$\sum_{j=1}^{n} x_{jk} = 1, \quad k = 1, \ldots, n \tag{3}$$

$$C_1^1 = \sum_{j=1}^{n} a_j x_{j1} \tag{4}$$

$$C_k^1 = C_{k-1}^1 + \sum_{j=1}^{n} a_j x_{jk}, \quad k = 2, \ldots, n \tag{5}$$

$$C_1^2 = C_1^1 + \sum_{j=1}^{n} b_j x_{j1} \tag{6}$$

$$C_k^2 \geq C_k^1 + \sum_{j=1}^{n} b_j x_{jk}, \quad k = 2, \ldots, n \tag{7}$$

$$C_k^2 \geq C_{k-1}^2 + \sum_{j=1}^{n} b_j x_{jk}, \quad k = 2, \ldots, n \tag{8}$$

$$C_j \geq C_k^2 - M(1 - x_{jk}), \quad j = 1, \ldots, n, \ k = 1, \ldots, n \tag{9}$$

$$Z \geq w_j C_j, \quad j = 1, \ldots, n \tag{10}$$

$$x_{jk} \in \{0, 1\}, \tag{11}$$

$$C_k^1, C_k^2, C_j \geq 0 \tag{12}$$

Constraint (2)–(3) ensure a one-to-one assignment between jobs and positions. Constraints (4)–(5) compute the completion time of each position on M_1. Constraints (6)–(8) compute the completion time of each position on M_2: for $k \geq 2$, the job at position k starts on M_2 only after it completes M_1 and the preceding job releases M_2. Constraint (9) links the position-based completion times to the job-based completion times via a big-M formulation: when $x_{jk} = 1$, job J_j is at position k and $C_j \geq C_k^2$; when $x_{jk} = 0$, the right-hand side becomes $C_k^2 - M \leq 0$, which is non-binding. Constraint (10) enforces $Z \geq w_j C_j$ for every job, so that minimizing Z yields the optimal $WC_{\max}$. Constraint (11) requires the assignment variables x_{jk} to be binary. Constraint (12) imposes the nonnegativity of the completion times C_k^1, C_k^2 and C_j.

3 Problem formulation

Theorem 3.1. *$F2 \parallel WC_{\max}$ is NP-hard.*

Proof. We reduce from the Partition problem: given n positive integers $x_1, x_2, \ldots, x_r$ with $\sum_{i=1}^{r} x_i = 2T$, does there exist subset Z_1 and $Z_2 \subseteq \{1, \ldots, r\}$ such that: $\sum_{i \in Z_1} x_i = \sum_{i \in Z_2} x_i = T$?

Construct an instance of $F2 \parallel WC_{\max}$, Let the number of jobs $n = r + 1$. Let $a_j = 1$, $b_j = rx_j$, $w_j = T + 1$ for jobs $j = 1, \ldots, n-1$. Put $a_n = rT$, $b_n = 1$, $w_n = 2T + 1$ for job J_n. Does there exist a threshold Y such that:

$$WC_{\max} \leq Y = r(T + 1)(2T + 1)$$

holds?

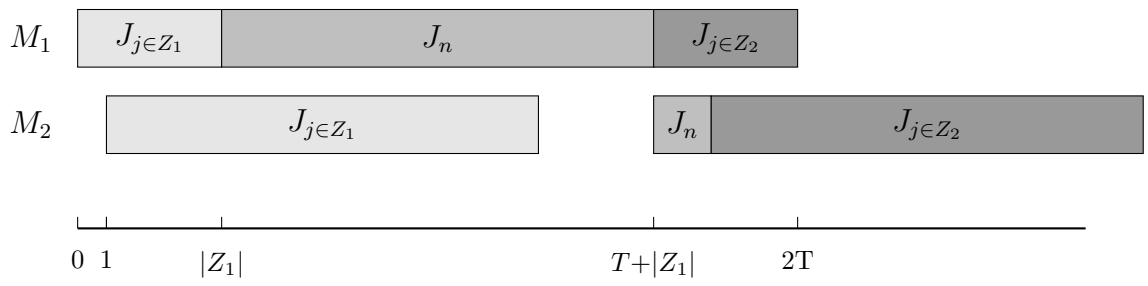

Figure 3-1

We partition jobs j_1 to j_{n-1} into two subsets via j_n, the distribution of jobs on the machine is shown in Figure 3. Since the processing times of A_1 and A_2 are unknown, while their total processing time is 2T. We may assume that the processing time of A_1 on M_2 is $r(T+\xi)$, A_2 is $r(T-\xi)$, where $-T \leq \xi \leq T$. Therefore, the completion time of J_n is determined by the following formula:

$$C_n^1 = |A_1| + rT + 1 \tag{13}$$

$$C_n^2 = 1 + r(T+\xi) + 1 \tag{14}$$

and $C_n = \max\{C_n^1, C_n^2\}$

If $\xi = 0$, then $\sum_{j\epsilon A_1} b_j = \sum_{j\epsilon A_2} b_j$, which means the partition problem has a feasible solution. In this case, the completion time of job J_n is determined by (13), we have:

$$\begin{aligned} C_n &= |A_1| + rT + 1 \leqslant (rT + r) = r(T+1) \\ w_n C_n &= (2T+1)(r(T+1)) = Y \\ WC_{\max} &= r(T+1)(C_n + rT) = Y \end{aligned}$$

so $w_j C_j \leqslant WC_{\max} = Y$

If the scheduling problem exists a threshold y such that $WC_{\max} \leq y$. When $\xi > 0$, $C_n$is determined by (14), we have:

$$\begin{aligned} C_n &= (1 + r(T+\xi) + 1) \geqslant (1 + r(T+1) + 1) \\ w_n C_n &\geqslant (2T+1)(1 + r(T+1) + 1) = Y \end{aligned}$$

When $\xi < 0$, $C_n$is determined by (13), we have:

$$\begin{aligned} C_{\max} &= (C_n + \sum_{j\epsilon A_2} b_j) \\ &= (1 + rT + |A_1| + r(T-\xi)) \\ &\geqslant (rT + r + rT) \\ &= r(2T+1) = Y \\ WC_{\max} &\geqslant r(T+1)(r(2T+1)) = Y \end{aligned}$$

so $WC_{\max} > Y$, there exists a contradiction.

Therefore, the Partition instance admits a solution if and only if the constructed $F2 \parallel WC_{\max}$ instance admits a schedule with $WC_{\max} \leq Y$. □

Theorem 3.2. *$F2 \parallel WC_{\max}$ is strongly NP-hard.*

Proof. We reduce from the strongly NP-complete 3-Partition problem: given $3m$ positive integers $x_1, \dots, x_{3m}$ and an integer B satisfying $B/4 < x_i < B/2$ and $\sum_{i=1}^{3m} x_i = mB$, can the integers be partitioned into m triples, each of sum B?

Let $R = 3m + 5$, $D = RB + 4$, and $T = mD$. We construct an instance with $3m$ element jobs $J_1, \dots, J_{3m}$ and m separator jobs $S_1, \dots, S_m$:

- for every element job J_j, set $a_j = 1$, $b_j = Rx_j + 1$;
- for every separator job S_k, set $a_{s_k} = RB + 1$, $b_{s_k} = 1$.

All processing times are positive integers. For $k = 1, \dots, m$, define $d_k = kD + 1$ and let $Q = (T + 2)^3$. Set

$$w_{s_k} = \left\lfloor \frac{Q}{d_k} \right\rfloor, \qquad w_j = \left\lfloor \frac{Q}{T} \right\rfloor \quad (j = 1, \dots, 3m),$$

and use the decision threshold $Y = Q$. Since $d_k \le T + 1$ and $Q \ge d_k(d_k + 1)$, we have $\lfloor Q/d_k \rfloor > d_k$. Thus, for every $d \in \{d_1, \dots, d_m, T\}$ and every integer completion time C,

$$\left\lfloor \frac{Q}{d} \right\rfloor C \le Q \quad \Longrightarrow \quad C \le d. \tag{15}$$

Indeed, if $C \ge d + 1$, write $Q = qd + r$ with $q = \lfloor Q/d \rfloor$ and $0 \le r < d$. Then $q(d + 1) = Q - r + q > Q$ because $q > d > r$. Suppose first that the 3-Partition instance is a yes-instance. Let $G_1, \dots, G_m$ be triples with $\sum_{J_j \in G_k} x_j = B$ for every k, and schedule

$$G_1, S_1, G_2, S_2, \dots, G_m, S_m.$$

The three element jobs in G_k use 3 units on M_1, and S_k uses $RB + 1$ units. Hence S_k completes on M_1 at time kD. The three element jobs in G_k use $RB + 3$ units on M_2; since the preceding separator completes on M_2 at time $(k - 1)D + 1$, the last element of G_k completes at time kD. Therefore

$$C_{s_k} = kD + 1 = d_k.$$

Every element job completes on M_2 no later than $T = mD$. By the definition of the weights, all weighted completion times are at most Q, so $WC_{\max} \le Y$.

Conversely, suppose that a schedule π satisfies $WC_{\max} \le Q$. By the standard permutation-schedule property for two-machine flow shops with a regular objective, it suffices to consider permutation schedules. The separator jobs have identical processing times. Therefore, if their positions are fixed, exchanging the labels of two separators that occur in the wrong order can only decrease the maximum weighted completion time, because $w_{s_1} \ge w_{s_2} \ge \cdots \ge w_{s_m}$. We may consequently assume that the separators occur in the order $S_1, S_2, \dots, S_m$. By (15),

$$C_{s_k} \le kD + 1 \quad (k = 1, \dots, m), \qquad C_j \le T \quad (j = 1, \dots, 3m).$$

No element job can occur after S_m. Indeed, if at least one element job did so, consider the last such element job. By the permutation-schedule property invoked above, the processing order on M_1 and M_2 coincides, so this job is also the last job on M_1; hence all m separators and all $3m$ element jobs have completed on M_1 no later than this job, and its completion on M_1 is at least the total M_1 processing time

$$m(RB+1)+3m = m(RB+4) = T.$$

Its completion on M_2 would consequently be strictly greater than T, contradicting $C_j \le T$. Thus the element jobs are partitioned into the m groups G_k lying between S_{k-1} and S_k, where S_0 denotes the beginning of the schedule. Put

$$B_k = \sum_{J_j \in G_k} x_j, \qquad E_k = \sum_{h=1}^{k} |G_h|, \qquad E_0 = 0.$$

For $k = 2, \dots, m$, the M_2 path gives

$$C_{s_k} \ge C_{s_{k-1}} + RB_k + |G_k| + 1,$$

while the M_1 path to S_{k-1} gives

$$C_{s_{k-1}} \ge (k-1)(RB+1) + E_{k-1} + 1.$$

Combining these inequalities with $C_{s_k} \le kD + 1$ yields

$$R(B_k - B) + E_k \le 3k, \qquad k = 2, \dots, m.$$

For $k = 1$, the M_2 path and $C_{s_1} \le D + 1$ similarly give

$$R(B_1 - B) + E_1 \le 4.$$

Because $R = 3m+5$, these inequalities imply $B_k \le B$ for every k: otherwise $B_k \ge B+1$ and the respective left-hand side is greater than $3k$ (or greater than 4 when $k = 1$). All element jobs belong to one of the groups, so $\sum_{k=1}^{m} B_k = mB$. Hence $B_k = B$ for every k. Finally, $B/4 < x_j < B/2$ implies that a group with sum B contains exactly three elements: at most two elements have sum strictly less than B, while at least four elements have sum strictly greater than B. Thus $G_1, \dots, G_m$ is a valid 3-partition.

The construction has $4m$ jobs, and all processing times, weights, and the threshold are bounded by a polynomial in m and B. Since 3-Partition remains NP-complete when B is polynomially bounded in m, this is a strong polynomial reduction. Therefore $F2 \parallel WC_{\max}$ is strongly NP-hard. □

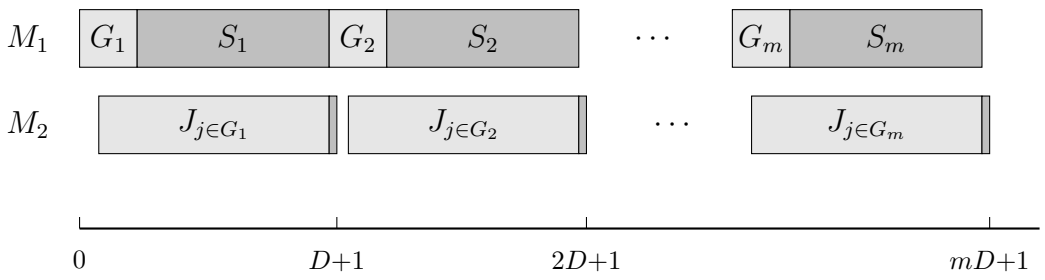

Figure 3-2: 3-Partition reduction

4 Algorithm

In this section, we design a dynamic programming algorithm for $F2 \parallel WC_{\max}$ and analyse its running time.

4.1 The number of weights is constant

For $F2 \parallel WC_{\max}$, an optimal schedule exists in which both machines process the jobs in the same order (Baker, 1974); we therefore restrict attention to such permutation schedules. Johnson's rule (Johnson, 1954) orders the n jobs so as to minimize the makespan: jobs with $a_j \le b_j$ are scheduled first in non-decreasing order of a_j, and the remaining jobs follow in non-increasing order of b_j.

For a given schedule π, Let $W_1 > W_2 > \cdots > W_K$ be the K distinct weights occurring among the jobs, and define $\mathcal{G}_k = \{J_j \in \mathcal{J} \mid w_j = W_k\}$, $k = 1, \ldots, K$. For $k = 1, \ldots, K$ in order, let ℓ_k be the position of the last job of weight W_k among the jobs not yet assigned to a block, i.e., among $\pi(\ell_{k-1}+1), \ldots, \pi(n)$, with the convention $\ell_k = \ell_{k-1}$ if no such job remains; set $\ell_0 = 0$. The blocks are then the segments

$$\mathcal{B}_k = \{\pi(\ell_{k-1}+1), \ldots, \pi(\ell_k)\}, \qquad k = 1, \ldots, K,$$

in particular, $\mathcal{B}_1 = \{\pi(1), \ldots, \pi(\ell_1)\}$. Equivalently, each block is removed from the schedule before the next one is formed, so the boundaries satisfy $\ell_1 \le \ell_2 \le \cdots \le \ell_K$ by construction and no block extends past its own last job. For every block $\mathcal{B}_i$, let A_i denote the total processing time of its jobs on machine M_1, B_i their total processing time on machine M_2, C_i the completion time of its last job on machine M_2, and L_i the weight of its last job. If $\ell_k = \ell_{k-1}$, then $\mathcal{B}_k$ contains no job and is called an empty block; an empty block $\mathcal{B}_i$ has $A_i = B_i = C_i = 0$ and may host any job of weight at most W_i. By construction, the last job of $\mathcal{B}_k$ carries weight W_k, so the block weights are non-increasing: $W_{\mathcal{B}_1} \ge W_{\mathcal{B}_2} \ge \cdots \ge W_{\mathcal{B}_K}$.

We illustrate the block construction with a small example.

Example 4.1. *Consider a 6-job instance with sequence $\pi = (J_1, \ldots, J_6)$ and the following (a_j, b_j, w_j) values:*

	J_1	J_2	J_3	J_4	J_5	J_6
a_j	2	5	6	8	4	3
b_j	5	7	9	4	3	2
w_j	2	2	5	2	4	2

The resulting groups are $\mathcal{G}_1 = \{J_3\}$, $\mathcal{G}_2 = \{J_5\}$ and $\mathcal{G}_3 = \{J_1, J_2, J_4, J_6\}$, and the blocks are $\mathcal{B}_1 = \{J_1, J_2, J_3\}$, $\mathcal{B}_2 = \{J_4, J_5\}$ and $\mathcal{B}_3 = \{J_6\}$.

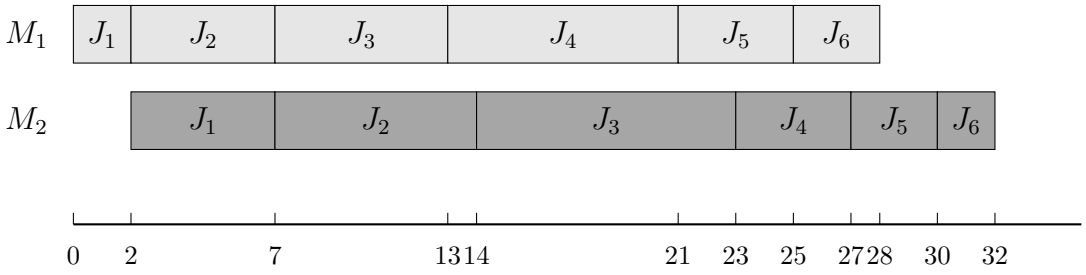

Figure 4.1

Lemma 4.1. *For any schedule π, reordering the jobs within each block according to Johnson's rule Johnson, 1954 and concatenating the resulting block schedules yields a schedule π' with $WC_{\max}(\pi') \leq WC_{\max}(\pi)$.*

Proof. By construction, the final job of block $\mathcal{B}_k$ carries weight $W(\mathcal{B}_k)$, and every job in $\mathcal{B}_k$ has weight at most $W(\mathcal{B}_k)$: a weight occurring in $\mathcal{B}_k$ either equals $W(\mathcal{B}_k)$, or it has already appeared in an earlier block with larger weight. Let $C_k(\sigma)$ denote the completion time of the last job of $\mathcal{B}_k$ under schedule σ.

We first show that, within each block, the maximum weighted completion time is attained by the last job. For any $J_j \in \mathcal{B}_k$, the last job of the block finishes no earlier than J_j, so $C_j(\sigma) \leq C_k(\sigma)$, and $w_j \leq W(\mathcal{B}_k)$ by the block construction. Hence $w_j C_j(\sigma) \leq W(\mathcal{B}_k) \cdot C_k(\sigma)$. Taking the maximum over $j \in \mathcal{B}_k$ yields $\max_{J_j \in \mathcal{B}_k} w_j C_j(\sigma) = W(\mathcal{B}_k) \cdot C_k(\sigma)$, and therefore

$$WC_{\max}(\sigma) = \max_{k=1,\ldots,K} \{W(\mathcal{B}_k) \cdot C_k(\sigma)\}.$$

Thus the overall objective depends only on the last job of each block.

Now let π' be the schedule obtained by reordering the jobs within each $\mathcal{B}_k$ according to Johnson's rule, while preserving the order of the blocks. We prove $C_k(\pi') \leq C_k(\pi)$ for all k by induction on k.

Base case $k = 1$. Block $\mathcal{B}_1$ starts at time zero on both machines regardless of its internal order, so Johnson's rule gives $C_1(\pi') \leq C_1(\pi)$.

Inductive step. Assume $C_{k-1}(\pi') \le C_{k-1}(\pi)$. On M_1, the start time of $\mathcal{B}_k$ equals the total M_1 work of $\mathcal{B}_1, \ldots, \mathcal{B}_{k-1}$, which is invariant under internal reordering, so $\mathcal{B}_k$ begins at the same instant on M_1 under both schedules. On M_2, $\mathcal{B}_k$ starts at $\max\{\sum_{i=1}^{k} A_i,\ C_{k-1}\}$, where the first term is identical under π and π', and the second is no larger under π' by the induction hypothesis. Hence $\mathcal{B}_k$ starts no later on M_2 under π' than under π. Together with the same M_1 start and the internally optimal Johnson ordering, this gives $C_k(\pi') \le C_k(\pi)$, completing the induction.

Finally,

$$WC_{\max}(\pi') = \max_k\ W(\mathcal{B}_k) \cdot C_k(\pi') \le \max_k\ W(\mathcal{B}_k) \cdot C_k(\pi) = WC_{\max}(\pi),$$

which establishes the lemma. □

4.2 Algorithm Properties and Dynamic Programming

We now develop a dynamic programming algorithm that processes jobs sequentially in Johnson order and assigns each job to one of the K blocks $\mathcal{B}_1, \ldots, \mathcal{B}_K$.

Lemma 4.1 provides the structural basis for the dynamic program: given any assignment of jobs to the K blocks, sequencing the jobs within each block $\mathcal{B}_i$ according to Johnson's rule and concatenating the blocks in non-increasing weight order yields a schedule whose $WC_{\max}$ value is no larger than that of any other schedule respecting the same assignment. For each job J_j ($j = 1, \ldots, n$), let $\mathscr{H}_j$ denote the set of states attainable after the first j jobs have been processed. A state records, for every block $\mathcal{B}_i$:

- A_i: the total processing time on machine M_1 of the jobs in block $\mathcal{B}_i$;
- B_i: the total processing time on machine M_2 of the jobs in block $\mathcal{B}_i$;
- C_i: the completion time of the last job of block $\mathcal{B}_i$ on machine M_2;
- L_i: the weight of the last job of block $\mathcal{B}_i$.

Since $W_1 > \cdots > W_K$, the blocks eligible for J_j form a prefix:

$$\mathcal{E}(J_j) = \{\, i : w_j \le W_i \,\}, \qquad i^* = \max_{i \in \{1,\ldots,K\}} \{\, i : W_i \ge w_j \,\},$$

so that $\mathcal{E}(J_j) = \{\, 1, \ldots, i^* \,\}$. A state is in fact the aggregate K-block tuple, described by $4K - 2$ parameters: A_K and B_K need not be stored, because after the first j jobs have been processed, each of these jobs belongs to exactly one block, so the totals $\sum_{i=1}^{K} A_i = \sum_{h=1}^{j} a_h$ and $\sum_{i=1}^{K} B_i = \sum_{h=1}^{j} b_h$ are known, and A_K and B_K are recovered by

$$A_K = \sum_{h=1}^{j} a_h - \sum_{i=1}^{K-1} A_i, \qquad B_K = \sum_{h=1}^{j} b_h - \sum_{i=1}^{K-1} B_i.$$

Algorithm 1: Block concatenation and evaluation

Input: State $\mathscr{H} = \{C_i, A_i, B_i, L_i\}$.

Step 1. [Initialization] Set $T_{A_0} \leftarrow 0$, $T_{B_0} \leftarrow 0$, $WC_{\max} \leftarrow 0$.
Step 2. [Evaluation] **for** $i = 1$ **to** K **do**
  $T_{A_i} \leftarrow T_{A_{i-1}} + A_i$
  $T_{B_i} \leftarrow \max\{\, T_{A_{i-1}} + C_i,\ T_{B_{i-1}} + B_i \,\}$
  if $\mathcal{B}_i \neq \emptyset$, *i.e.,* $A_i + B_i > 0$ **then**
    $WC_{\max} \leftarrow \max\{\, WC_{\max},\ L_i \cdot T_{B_i} \,\}$
  end
end

Output: $WC_{\max} = \max_i L_i \cdot T_{B_i}$.

Hence each state of $\mathscr{H}_j$ is represented by the tuple

$$(C_1, \ldots, C_K;\ A_1, \ldots, A_{K-1};\ B_1, \ldots, B_{K-1};\ L_1, \ldots, L_K).$$

The initial state set contains a single state with all components set to zero: $\mathscr{H}_0 = \{(0, \ldots, 0;\ 0, \ldots, 0;\ 0, \ldots, 0;\ 0, \ldots, 0)\}$. In Algorithm 1 below, T_{A_i} denotes the total M_1 processing time of the first i blocks, and T_{B_i} the completion time of block $\mathcal{B}_i$ on M_2.

Lemma 4.2. *Let* $\mathscr{H} = (C_1, \ldots, C_K; A_1, \ldots, A_{K-1}; B_1, \ldots, B_{K-1}; L_1, \ldots, L_K)$ *and* $\mathscr{H}' = (C'_1, \ldots, C'_K; A'_1, \ldots, A'_{K-1}; B'_1, \ldots, B'_{K-1}; L'_1, \ldots, L'_K)$ *be two states in* $\mathscr{H}_j$ *for job* J_j. *If* $A_i = A'_i$, $B_i = B'_i$, $L_i = L'_i$ *for all* i, *and* $C_i \leq C'_i$ *for all* i, *then* $\mathscr{H}'$ *can be discarded without affecting the optimal solution.*

Proof. Consider any continuation that extends state $\mathscr{H}'$ to a complete schedule Π'. Replacing the prefix represented by $\mathscr{H}'$ with the prefix represented by $\mathscr{H}$ yields a schedule Π that uses the same assignment of the remaining jobs. For every subsequent block $\mathcal{B}_k$ ($k \geq 2$), the M_2 start time is $\max\{\sum_{i=1}^{k} A_i,\ C_{k-1}\}$, where C_{k-1} is no larger under $\mathscr{H}$ than under $\mathscr{H}'$. By induction, the M_2 completion time of every subsequent block is no larger under $\mathscr{H}$, and since A_i, B_i and L_i are identical, the objective value $WC_{\max}$ of Π does not exceed that of Π'. □

For $j = 1, \ldots, n$, the algorithm extends $\mathscr{H}_{j-1}$ to $\mathscr{H}_j$ as follows: for each state in $\mathscr{H}_{j-1}$ and each block $i \in \mathcal{E}(J_j)$, the algorithm places J_j at the end of $\mathcal{B}_i$. Only the i-th slice of the state is updated, while all other blocks remain unchanged:

$$C'_i = \max\{C_i + b_j,\ A_i + a_j + b_j\}, \quad A'_i = A_i + a_j, \quad B'_i = B_i + b_j.$$

The resulting candidate states are collected in temporary sets $\mathcal{Y}_1, \ldots, \mathcal{Y}_K$. By Lemma 4.2, merging the candidates removes dominated and duplicate states and yields $\mathscr{H}_j$.

Algorithm 2 returns the optimal objective value Z^*; the optimal schedule itself is recovered by standard backtracking. To this end, each state in $\mathscr{H}_j$ stores a pointer to the state in

Algorithm 2: DP for $F2 \parallel WC_{\max}$

Input: Jobs $\mathcal{J}$ with (a_j, b_j, w_j); block count K; distinct weights $W_1 > \cdots > W_K$.

Step 1. [Preprocessing] Set $L_i \leftarrow W_i$ for $i = 1, \ldots, K$.
Step 2. [Initialization] Set $\mathscr{H}_0 \leftarrow \{(0,0,0,0)_{i=1}^{K}\}$.
Step 3. [Generation] **for** $j = 1$ **to** n **do**
    $\mathcal{Y}_i \leftarrow \emptyset$ for $i = 1, \ldots, K$
    for *each* $(\{C_i, A_i, B_i, L_i\}_{i=1}^{K}) \in \mathscr{H}_{j-1}$ **do**
        for $i = 1$ **to** K **do**
            if $w_j \le L_i$ **then**
                if $\mathcal{B}_i$ *is empty, i.e.,* $A_i = B_i = C_i = 0$ **then**
                    $C_i' \leftarrow a_j + b_j$
                else
                    $C_i' \leftarrow \max\{\, C_i + b_j,\ A_i + a_j + b_j \,\}$
                end
                $A_i' \leftarrow A_i + a_j$, $B_i' \leftarrow B_i + b_j$
                Add the updated state to $\mathcal{Y}_i$
            end
        end
    end
    [Merging] $\mathscr{H}_j \leftarrow \textsc{Merge}(\mathcal{Y}_1, \ldots, \mathcal{Y}_K)$
end
Step 4. [Result] Set $Z^* \leftarrow +\infty$
for *each state* $S = \{C_i, A_i, B_i, L_i\}_{i=1}^{K} \in \mathscr{H}_n$ **do**
    $WC_{\max} \leftarrow$ Algorithm $1\big((A_i, B_i, C_i, L_i)_{i=1}^{K}\big)$
    $Z^* \leftarrow \min\{\, Z^*,\ WC_{\max} \,\}$
end

Output: The optimal value Z^*; the optimal schedule is recovered by backtracking.

$\mathscr{H}_{j-1}$ from which it was derived, together with the block $i \in \mathcal{E}(J_j)$ into which J_j was placed. Starting from the state in $\mathscr{H}_n$ attaining Z^*, backtracking along these pointers recovers, for every job J_j, the block $\mathcal{B}_i$ it belongs to. Sequencing the jobs of each block according to Johnson's rule (Lemma 4.1) and concatenating the blocks then yields an optimal schedule π^* with $WC_{\max}(\pi^*) = Z^*$.

Theorem 4.1. *Algorithm 2 solves $F2 \parallel WC_{\max}$ optimally in $O\big(n \cdot K \cdot |\mathscr{H}|_{\max}\big)$ time, where $|\mathscr{H}|_{\max} \le K^K \cdot V^{3K-3}$, $V = \sum_{j=1}^{n} a_j + \sum_{j=1}^{n} b_j$. For constant K, this is $O(n \cdot V^{3K-3})$.*

Proof. Each state in $\mathscr{H}_j$ is a $(4K-2)$-tuple $(C_1, \ldots, C_K;\ A_1, \ldots, A_{K-1};\ B_1, \ldots, B_{K-1};\ L_1, \ldots, L_K)$. We bound the number of distinct values each component can attain.

The quantity A_i is the total M_1 processing time of jobs assigned to block $\mathcal{B}_i$, hence $A_i \in [0,\, V]$. Since the j jobs under consideration form a prefix of the fixed Johnson order, $\sum_{i=1}^{K-1} A_i$ equals the sum of the a_h values over that prefix. Consequently at most $K-1$ of the A_i are independent, contributing at most V^{K-1} distinct A-vectors. Symmetrically, $\sum_{i=1}^{K-1} B_i$ is the prefix sum of the b_j's, so the B_i components likewise contribute at most V^{K-1} distinct combinations.

The quantity C_i is the makespan of $\mathcal{B}_i$ when scheduled in isolation by Johnson's rule. Since $C_i \le A_i + B_i \le V$ (each block's total M_1 and M_2 work is bounded by V), the C-vector contributes at most V^{K-1} distinct values. The weight L_i is the weight of the last job in $\mathcal{B}_i$ and takes values from $\{W_1, \ldots, W_K\}$, yielding at most K^K distinct L-vectors.

Multiplying these bounds yields

$$|\mathscr{H}_j| \le K^K \cdot V^{K-1} \cdot V^{2K-2} = K^K \cdot V^{3K-3}.$$

In iteration j, the algorithm takes each state in $\mathscr{H}_{j-1}$, tries every block $i \in \{1, \ldots, K\}$, computes the updated tuple in $O(1)$ time, and inserts it into $\mathcal{Y}_i$. The merge step collects the K sets $\mathcal{Y}_1, \ldots, \mathcal{Y}_K$ and removes duplicates in time proportional to the total number of generated tuples. Hence iteration j costs $O(K \cdot |\mathscr{H}_{j-1}|)$, and summing over all n iterations gives $O(n \cdot K \cdot |\mathscr{H}|_{\max})$.

The final concatenation via Algorithm 1 evaluates each state in $\mathscr{H}_n$ in $O(K)$ time, which is dominated by the DP phase. When K is fixed, K^K is constant, giving $O(n \cdot V^{3K-3})$. □

4.3 A 2-Approximation Algorithm

The exact dynamic program of Section 4 runs in pseudo-polynomial time, and the FPTAS of Section 4.4 has running time exponential in the number K of distinct weights. When only a constant-factor guarantee is needed, the following elementary rule suffices.

Theorem 4.2. *Algorithm 3 runs in $O(n \log n)$ time and returns a schedule with $WC_{\max} \le 2\, WC^*_{\max}$.*

Algorithm 3: 2-Approximation for $F2 \parallel WC_{\max}$

Input: Jobs $\mathcal{J}$ with (a_j, b_j, w_j).

1 **Step 1.** Sort the jobs by non-increasing weight w_j (ties broken arbitrarily).

2 **Step 2.** Schedule the jobs in this order on both machines.

Output: The resulting permutation schedule, with $WC_{\max} \le 2\,WC^*_{\max}$.

Proof. Let σ be the schedule returned by Algorithm 3, let j_k be the job in position k of σ, and let $S_k = \{j_1, \dots, j_k\}$. Since the jobs are sorted by non-increasing weight, every job of S_k has weight at least w_{j_k}.

We first bound the completion times in σ. By the standard two-machine flow-shop recurrence, the job in position k completes on M_2 at time

$$C_{j_k}(\sigma) = \max_{1 \le i \le k} \Big\{ \sum_{h=1}^{i} a_{j_h} + \sum_{h=i}^{k} b_{j_h} \Big\} \le \sum_{h \in S_k} \big(a_h + b_h\big),$$

and therefore

$$WC_{\max}(\sigma) \le \max_{1 \le k \le n} w_{j_k} \sum_{h \in S_k} \big(a_h + b_h\big).$$

It remains to bound the right-hand side by $2WC^*_{\max}$. Fix a prefix S_k and let π^* be an optimal schedule. Let u be the job of S_k that finishes last on M_1 in π^*, and v the job of S_k that finishes last on M_2. All jobs of S_k are processed on M_1 before u completes M_1, so $C^*_u \ge \sum_{h \in S_k} a_h$; likewise, all M_2 processing of the jobs in S_k precedes v's completion on M_2, so $C^*_v \ge \sum_{h \in S_k} b_h$. Since $u, v \in S_k$, we have $w_u, w_v \ge w_{j_k}$, and

$$WC^*_{\max} \ge w_{j_k} \max\Big\{ \sum_{h \in S_k} a_h,\ \sum_{h \in S_k} b_h \Big\} \ge \frac{w_{j_k}}{2} \Big(\sum_{h \in S_k} a_h + \sum_{h \in S_k} b_h \Big),$$

where the last inequality uses $\max\{x, y\} \ge (x + y)/2$. Hence $w_{j_k} \sum_{h \in S_k} (a_h + b_h) \le 2WC^*_{\max}$ for every k, and the claim follows from the upper bound above. □

4.4 FPTAS

We now convert the exact dynamic program of Algorithm 2 into a fully polynomial-time approximation scheme by geometric rounding of its time coordinates: the completion time C_i of block $\mathcal{B}_i$ and its total processing times A_i, B_i on M_1 and M_2 are rounded. The block weights $W_1 > \cdots > W_K$, stored as the labels $L_i = W_i$, are fixed and are not rounded.

Fix $0 < \varepsilon < 1$ and let

$$\delta = 1 + \frac{\varepsilon}{2n}, \qquad v = \Big\lceil \log_\delta V \Big\rceil,$$

where $V = \sum_{j=1}^{n} a_j + \sum_{j=1}^{n} b_j$. For every block $\mathcal{B}_i$, the recurrence

$$C'_i = \max\{C_i + b_j,\ A_i + a_j + b_j\}, \qquad A'_i = A_i + a_j, \qquad B'_i = B_i + b_j$$

preserves $A_i, B_i \le V$ and $C_i \le A_i + B_i \le V$, so each of the A, B and C dimensions ranges over $[0, V]$. We split the range $[0, V]$ of each of the three dimensions into the $v+1$ intervals

$$I_0 = [0, 1), \qquad I_r = [\delta^{r-1}, \delta^r) \ \ (1 \le r \le v-1), \qquad I_v = [\delta^{v-1}, V],$$

using the same family $\{I_r\}$ for the A, B and C dimensions. All processing times are integral, so every value taken by a coordinate C_i, A_i, B_i is a nonnegative integer. Hence any two values lying in a common interval satisfy $x \le \delta\, y$; we call this the *interval-dominance* property, and it is the only fact about the rounding used in the analysis below.

For each block $\mathcal{B}_i$, let $r_i^C, r_i^A, r_i^B \in \{0, \dots, v\}$ be the indices of the intervals containing C_i, A_i and B_i, respectively.

A *box* is the set of states whose time coordinates fall in a fixed tuple of intervals. Collecting the per-block indices into the tuples $\mathbf{r}^C = (r_1^C, \dots, r_K^C)$, $\mathbf{r}^A = (r_1^A, \dots, r_K^A)$ and $\mathbf{r}^B = (r_1^B, \dots, r_K^B)$, each in $\{0, \dots, v\}^K$, the box is

$$\mathscr{B}(\mathbf{r}^C; \mathbf{r}^A; \mathbf{r}^B) = \Big\{ (C_i, A_i, B_i, L_i)_{i=1}^K : \ C_i \in I_{r_i^C},\ A_i \in I_{r_i^A},\ B_i \in I_{r_i^B} \Big\}.$$

There are $(v+1)^{3K}$ boxes, a number independent of the job sizes. The scheme stores at most one *representative* state per box; since the weight labels $L_i = W_i$ are fixed, they are common to all states of a box.

Theorem 4.3. *For every $0 < \varepsilon < 1$, Algorithm 4 returns a schedule with*

$$WC_{\max} \ \le \ (1+\varepsilon)\, WC^*_{\max},$$

where $WC^*_{\max}$ is the optimal value of $F2 \parallel WC_{\max}$.

Theorem 4.4. *For fixed K, Algorithm 4 runs in*

$$O\!\left(n^{3K+1} \cdot K \cdot \frac{(\log V)^{3K}}{\varepsilon^{3K}} \right)$$

time and is therefore a fully polynomial-time approximation scheme for $F2 \parallel WC_{\max}$.

Proof. At each level j, Algorithm 4 stores at most one representative per box, and there are $(v+1)^{3K}$ boxes. Each representative generates K candidates in $O(1)$ time, and the final evaluation of the representatives at level n costs $O(K)$ per box. Hence the running time is

$$O\big(nK\,(v+1)^{3K}\big).$$

Since $\ln(1+\varepsilon/(2n)) \ge \varepsilon/(4n)$ for $0 < \varepsilon < 1$,

$$v = \left\lceil \frac{\ln V}{\ln \delta} \right\rceil = O\!\left(\frac{n \log V}{\varepsilon} \right),$$

and for constant K the bound reduces to $O(n^{3K+1} K (\log V)^{3K} / \varepsilon^{3K})$, polynomial in n, $\log V$, and $1/\varepsilon$. $\square$

Algorithm 4: Boxed FPTAS for $F2 \parallel WC_{\max}$

Input: Jobs $\mathcal{J}$ with (a_j, b_j, w_j) in Johnson order; block count K; accuracy ε.

Step 1. [Preprocessing] Set $\delta = 1 + \varepsilon/(2n)$, $v = \lceil \log_\delta V \rceil$, and the intervals I_r; set $L_i \leftarrow W_i$ for $i = 1, \ldots, K$.

Step 2. [Initialization] Store the zero state as the representative of $\mathscr{B}(\mathbf{0}; \mathbf{0}; \mathbf{0})$.

Step 3. [Generation] **for** $j = 0$ **to** $n - 1$ **do**

  for *each box* $\mathscr{B}$ *with representative* $s = (C_i, A_i, B_i, L_i)_{i=1}^{K}$ **do**

    for $i = 1$ **to** K **do**

      if $w_{j+1} \le L_i$ **then**

        $\tilde{C}_i = \max\{C_i + b_{j+1},\ A_i + a_{j+1} + b_{j+1}\}$, $\tilde{A}_i = A_i + a_{j+1}$, $\tilde{B}_i = B_i + b_{j+1}$

        Map $\tilde{s}$ to its box $\mathscr{B}'$

        if $\mathscr{B}'$ *has no representative* **then**

          Store $\tilde{s}$ and record s as its predecessor

        else

          Keep either one of the two representatives of $\mathscr{B}'$

        end

      end

    end

  end

end

Step 4. [Result] Evaluate every representative at level n with Algorithm 1

Output: The schedule attaining the minimum $WC_{\max}$, with $WC_{\max} \le (1 + \varepsilon)\, WC^*_{\max}$.

Algorithm 5: NEH heuristic for $F2 \parallel WC_{\max}$

Input: Jobs $\mathcal{J}$ with (a_j, b_j, w_j).

Step 1. [Ordering] Sort the jobs by non-increasing weight w_j, breaking ties by non-increasing $a_j + b_j$; let $J_{[1]}, \dots, J_{[n]}$ be the resulting order.

Step 2. [Initialization] $\sigma \leftarrow (J_{[1]})$.

Step 3. [Insertion] **for** $k = 2$ **to** n **do**

  $\sigma \leftarrow$ the sequence obtained by inserting $J_{[k]}$ into σ at a position minimizing $WC_{\max}$.

end

Output: The permutation schedule σ.

5 Heuristic Algorithms

The exact dynamic program of Section 4 is pseudo-polynomial, and the running time of the FPTAS of Section 4.4 is exponential in the number K of distinct weights. For large instances, fast heuristics are therefore of practical interest. In this section we present two heuristics for $F2 \parallel WC_{\max}$: an adaptation of the NEH algorithm (Nawaz et al., 1983) and an ant colony optimization (ACO) algorithm (Dorigo et al., 1996).

Throughout this section, the objective of a permutation $\sigma = (\sigma(1), \dots, \sigma(k))$ of k jobs is evaluated by the standard two-machine flow-shop recurrence: the job in position t completes on M_2 at time

$$C_{\sigma(t)} = \max_{1 \le i \le t} \Big\{ \sum_{h=1}^{i} a_{\sigma(h)} + \sum_{h=i}^{t} b_{\sigma(h)} \Big\}, \qquad t = 1, \dots, k,$$

so that

$$WC_{\max}(\sigma) = \max_{1 \le t \le k} w_{\sigma(t)}\, C_{\sigma(t)},$$

which is computed in $O(k)$ time.

5.1 An NEH-type heuristic

The NEH algorithm (Nawaz et al., 1983) is one of the best-performing constructive heuristics for flow-shop scheduling. It orders the jobs by decreasing priority and then inserts them one by one into the partial schedule at the position that minimizes the objective.

For $F2 \parallel WC_{\max}$, the natural priority is the weight: jobs of large weight should be scheduled early, because each job contributes to the objective through the product $w_j C_j$. We therefore sort the jobs by non-increasing weight, breaking ties by non-increasing total processing time $a_j + b_j$. The k-th job is inserted into each of the k possible positions of the current partial schedule, and a position attaining the smallest $WC_{\max}$ is kept.

Proposition 5.1. *Algorithm 5 runs in $O(n^3)$ time.*

Proof. At step k, the job $J_{[k]}$ is inserted into each of the k positions of a partial schedule of $k-1$ jobs, and each candidate sequence is evaluated in $O(k)$ time by the recurrence above. Hence step k costs $O(k^2)$, and the total running time is $O\big(\sum_{k=1}^{n} k^2\big) = O(n^3)$. □

Remark 5.1. *The initial ordering by non-increasing weight coincides with the rule of the 2-approximation algorithm of Section 4.3. The insertion step then repairs the local violations of the weight order that may be caused by the processing times.*

5.2 An ant colony optimization algorithm

Ant colony optimization (Dorigo et al., 1996) is a metaheuristic in which artificial ants construct solutions incrementally, guided by pheromone trails that encode the experience accumulated in previous iterations. We adapt it to $F2 \parallel WC_{\max}$ as follows.

Pheromone model. A pheromone value τ_{ij} is attached to the assignment of job J_j to position i, for all $i, j = 1, \ldots, n$. All entries are initialized to $\tau_{ij} = 1/\big(n \cdot WC_{\max}(\sigma^{\mathrm{NEH}})\big)$, where σ^{NEH} is the solution produced by Algorithm 5.

Solution construction. Each ant builds a permutation position by position. At position i, the ant selects an unscheduled job J_j with probability

$$p_{ij} = \frac{[\tau_{ij}]^{\alpha}\,[\eta_j]^{\beta}}{\sum_{h \in \mathcal{U}} [\tau_{ih}]^{\alpha}\,[\eta_h]^{\beta}},$$

where $\mathcal{U}$ is the set of jobs not yet scheduled, the parameters $\alpha, \beta \geq 0$ control the relative influence of the pheromone trail and of the heuristic information, and the heuristic information $\eta_j = w_j/(a_j + b_j)$ favors jobs of high weight and small total processing time.

Local search. Each constructed solution is improved by one pass of best-insertion local search, in which every job is removed in turn and reinserted at the position that most decreases the objective.

Pheromone update. After all ants of an iteration have completed their solutions, the pheromone evaporates on every assignment,

$$\tau_{ij} \leftarrow (1-\rho)\,\tau_{ij}, \qquad i, j = 1, \ldots, n,$$

where $\rho \in (0, 1)$ is the evaporation rate, and the iteration-best solution σ^{ib} deposits pheromone on the assignments it uses,

$$\tau_{ij} \leftarrow \tau_{ij} + \frac{\rho}{WC_{\max}(\sigma^{\mathrm{ib}})} \qquad \text{for every assignment } (i, j) \text{ of } \sigma^{\mathrm{ib}}.$$

Proposition 5.2. *Each iteration of Algorithm 6 runs in $O(m\,n^3)$ time.*

Proof. Building one permutation takes $O(n^2)$ time, since each of the n positions considers at most n candidate jobs. One pass of best-insertion local search evaluates, for each of the

Algorithm 6: Ant colony optimization for $F2 \parallel WC_{\max}$

Input: Jobs $\mathcal{J}$ with (a_j, b_j, w_j); parameters $\alpha, \beta \geq 0$, $0 < \rho < 1$, the number of ants m, and the number of iterations T.

Step 1. [Initialization] Compute σ^{NEH} by Algorithm 5; set $\sigma^* \leftarrow \sigma^{\mathrm{NEH}}$ and $\tau_{ij} \leftarrow 1/\big(n \cdot WC_{\max}(\sigma^{\mathrm{NEH}})\big)$ for all i, j.
Step 2. [Iteration] **for** $t = 1$ **to** T **do**
  for *each ant* $q = 1$ **to** m **do**
    Build a permutation σ_q by the probabilistic rule with probabilities p_{ij}
    Apply one pass of best-insertion local search to σ_q
  end
  Let σ^{ib} be the solution with the smallest $WC_{\max}$ among the m ants
  $\tau_{ij} \leftarrow (1 - \rho)\,\tau_{ij}$ for all i, j
  $\tau_{ij} \leftarrow \tau_{ij} + \rho / WC_{\max}(\sigma^{\mathrm{ib}})$ for each assignment (i, j) of σ^{ib}
  if $WC_{\max}(\sigma^{\mathrm{ib}}) < WC_{\max}(\sigma^*)$ **then**
    $\sigma^* \leftarrow \sigma^{\mathrm{ib}}$
  end
end

Output: The best permutation σ^* found.

n jobs, at most n insertion positions, each in $O(n)$ time, and thus costs $O(n^3)$ per ant. Summing over the m ants gives $O(m\,n^3)$ per iteration. □

Remark 5.2. *ACO carries no worst-case performance guarantee. In the flow-shop scheduling literature it typically returns better solutions than constructive heuristics such as NEH, at the price of a much higher running time, and its performance depends on the parameter settings, for which common choices are* $\alpha = 1$, $\beta = 2$ *and* $\rho = 0.1$.

6 Conclusion

References

Baker, K. R. (1974). *Introduction to Sequencing and Scheduling.* Wiley, New York.

Dorigo, M., Maniezzo, V., and Colorni, A. (1996). Ant system: optimization by a colony of cooperating agents. *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, 26(1):29–41.

Hall, L. A. (1994). A polynomial approximation scheme for a constrained flow-shop scheduling problem. *Mathematics of Operations Research*, 19(1):68–85.

Johnson, S. M. (1954). Optimal two- and three-stage production schedules with setup times included. *Naval Research Logistics Quarterly*, 1(1):61–68.

Nawaz, M., Enscore, E. E., and Ham, I. (1983). A heuristic algorithm for the m-machine, n-job flow-shop sequencing problem. *Omega*, 11(1):91–95.